EN3160 Assignment 01

T.S.R.Peiris- 230470U

git repo - https://github.com/angstrom10/EN3160---Image-Processing-and-Machine-Vision

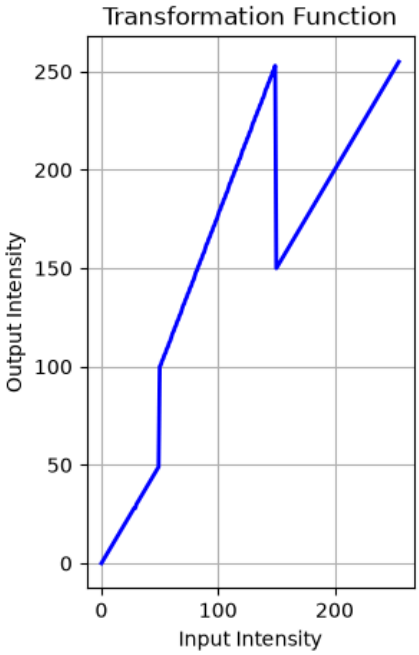
Question 1

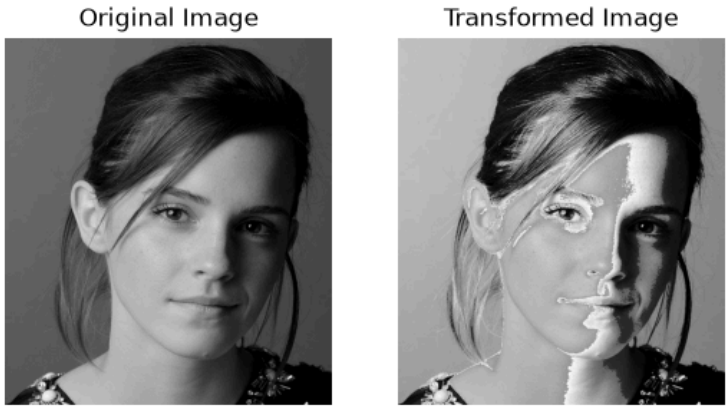
In [261…

```
import cv2 as cv
import matplotlib.pyplot as plt
import numpy as np
f = cv.imread('images/emma.jpg', cv.IMREAD_GRAYSCALE)
def transform(breakpoints):
    t = np.zeros(256, dtype=np.uint8)
    for i in range(len(breakpoints) - 1):
        x1, y1 = breakpoints[i]
        x2, y2 = breakpoints[i + 1]
        if x1 == x2:
            t[x1] = y1
        else:
            for x in range(x1, x2 + 1):
                t[x] = y1 + (y2 - y1) * ((x - x1) / (x2 - x1))
    return t
points = [
    (0, 0),
    (50, 50),
    (50, 100),
    (150, 255),
    (150, 150),
    (255, 255)
]
t = transform(points)
```

In [262…

```
fig, ax = plt.subplots(1, 3, figsize=(10, 5))
ax[0].plot(np.arange(256), t, 'b-', linewidth=2)
ax[0].grid()
ax[0].set_title('Transformation Function')
ax[0].set_xlabel('Input Intensity')
ax[0].set_ylabel('Output Intensity')
ax[1].imshow(f, cmap='gray')
ax[1].set_title('Original Image')
ax[1].axis('off')
f_transformed = cv.LUT(f, t)
ax[2].imshow(f_transformed, cmap='gray')
ax[2].set_title('Transformed Image')
ax[2].axis('off')
plt.show()
```





The transformation points were selected to increase mid intensity pixel values.

Question 2

In [263…

```
f = cv.imread('images/brain.jpeg')
white_points = [(0, 0), (145, 20), (185, 80), (225, 255),(255, 255)]
gray_points = np.array([(0, 0),(100, 40),(150, 255),(210, 40),(255, 0)])

white_t = transform(white_points)
gray_t  = transform(gray_points)

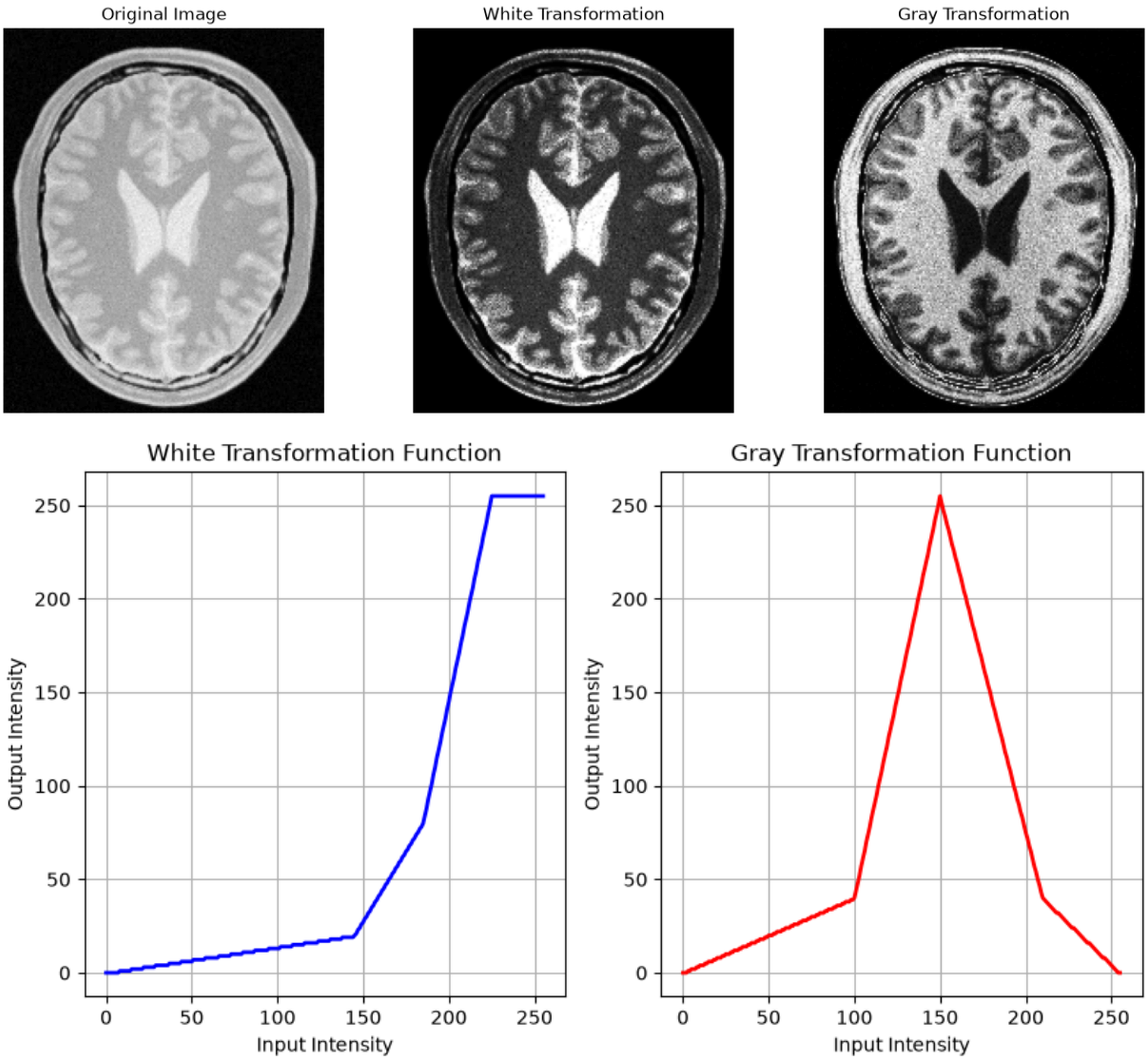
white_transformed = cv.LUT(f, white_t)
gray_transformed  = cv.LUT(f, gray_t)
```

In [264…

```
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(f)
ax[0].set_title('Original Image')
ax[0].axis('off')
ax[1].imshow(white_transformed)
ax[1].set_title('White Transformation')
ax[1].axis('off')
ax[2].imshow(gray_transformed)
ax[2].set_title('Gray Transformation')
ax[2].axis('off')
fig,ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].plot(np.arange(256), white_t, 'b-', linewidth=2)
ax[0].set_title('White Transformation Function')
ax[0].grid()
ax[0].set_xlabel('Input Intensity')
ax[0].set_ylabel('Output Intensity')
ax[1].plot(np.arange(256), gray_t, 'r-', linewidth=2)
ax[1].set_title('Gray Transformation Function')
ax[1].grid()
```

```
ax[1].set_xlabel('Input Intensity')
ax[1].set_ylabel('Output Intensity')
```

Out[264… `Text(0, 0.5, 'Output Intensity')`



White matter: The intensity transformation increases the contrast in the brighter regions of the brain, making the white matter more distinguishable from surrounding tissue.

Gray matter: The transformation enhances the intensity range associated with gray matter. The breakpoints were adjusted to avoid excessive saturation and preserve the visibility of other brain structures.

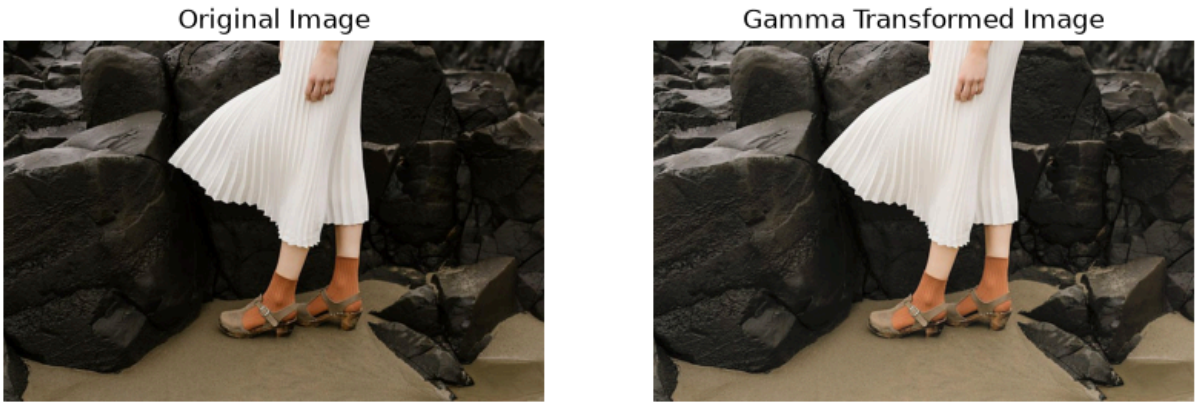
Question 3

In [265…

```
f= cv.imread('images/gamma.jpeg')
L,a,b = cv.split(cv.cvtColor(f, cv.COLOR_BGR2Lab))
gamma = 0.75
L=np.array(255 * (L / 255) ** gamma, dtype=np.uint8)
gamma_transformed = cv.cvtColor(cv.merge((L,a,b)), cv.COLOR_Lab2BGR)
```

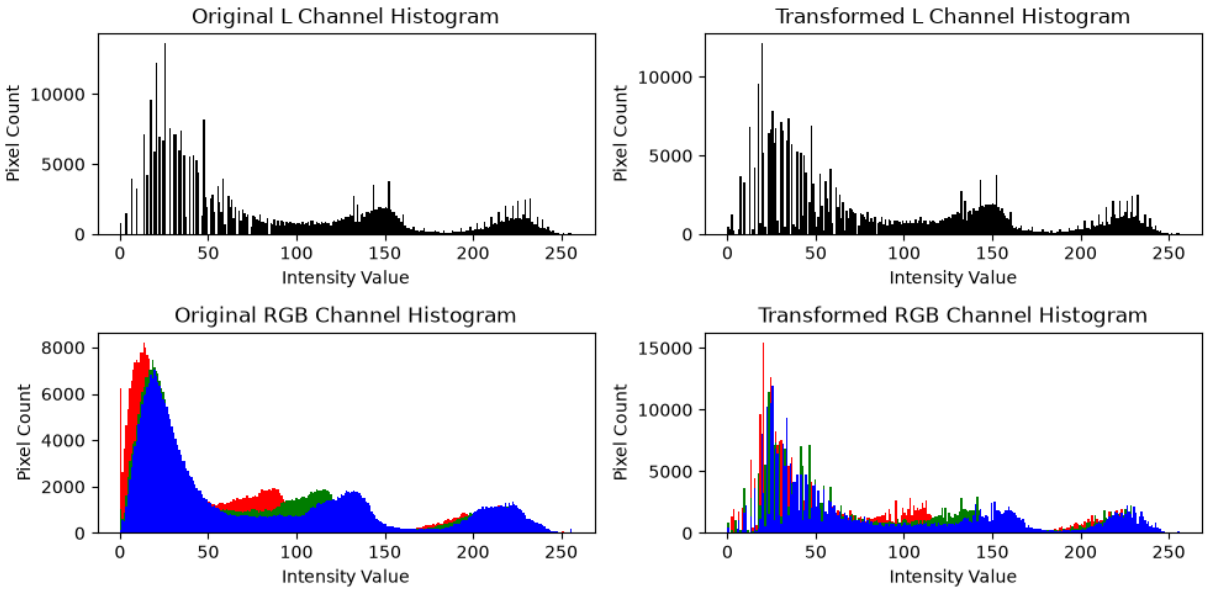
In [266…
```
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
ax[0].set_title('Original Image')
ax[0].axis('off')
ax[1].imshow(cv.cvtColor(gamma_transformed, cv.COLOR_BGR2RGB))
ax[1].set_title('Gamma Transformed Image')
ax[1].axis('off')
```

Out[266…
```
(np.float64(-0.5), np.float64(719.5), np.float64(479.5), np.float64(-0.5))
```



In [267…
```
L_transformed = cv.split(cv.cvtColor(gamma_transformed, cv.COLOR_BGR2Lab))[0]
R,G,B = cv.split(f)
R_transformed,G_transformed,B_transformed = cv.split(gamma_transformed)
```

In [268…
```
fig, ax = plt.subplots(2, 2, figsize=(10, 5))
ax[0, 0].hist(L.ravel(), bins=256, range=(0, 256), color='black')
ax[0, 0].set_title('Original L Channel Histogram')
ax[0, 1].hist(L_transformed.ravel(), bins=256, range=(0, 256), color='black')
ax[0, 1].set_title('Transformed L Channel Histogram')
ax[1, 0].hist(R.ravel(), bins=256, range=(0, 256), color='red')
ax[1, 0].set_title('Original R Channel Histogram')
ax[1, 1].hist(R_transformed.ravel(), bins=256, range=(0, 256), color='red')
ax[1, 1].set_title('Transformed R Channel Histogram')
ax[1, 0].hist(G.ravel(), bins=256, range=(0, 256), color='green')
ax[1, 0].set_title('Original G Channel Histogram')
ax[1, 1].hist(G_transformed.ravel(), bins=256, range=(0, 256), color='green')
ax[1, 1].set_title('Transformed G Channel Histogram')
ax[1, 0].hist(B.ravel(), bins=256, range=(0, 256), color='blue')
ax[1, 0].set_title('Original RGB Channel Histogram')
ax[1, 1].hist(B_transformed.ravel(), bins=256, range=(0, 256), color='blue')
ax[1, 1].set_title('Transformed RGB Channel Histogram')
ax[0, 0].set_xlabel('Intensity Value')
ax[0, 0].set_ylabel('Pixel Count')
ax[0, 1].set_xlabel('Intensity Value')
ax[0, 1].set_ylabel('Pixel Count')
ax[1, 0].set_xlabel('Intensity Value')
ax[1, 0].set_ylabel('Pixel Count')
ax[1, 1].set_xlabel('Intensity Value')
ax[1, 1].set_ylabel('Pixel Count')
plt.tight_layout()
```



The histogram of the corrected image appears noisier because gamma correction is a non-linear operation. When applied to the L plane, which represents lightness in the LAB colour space, it can amplify small differences in pixel intensity. Since pixel values are discrete, the non-linear transformation redistributes them unevenly, causing quantisation noise and a noisier histogram.

Question 4

In [269…
```
f= cv.imread('images/spider_man.jpeg')
H,S,V = cv.split(cv.cvtColor(f, cv.COLOR_BGR2HSV))
```

In [270…
```
fig, ax = plt.subplots(1, 3, figsize=(10, 5))
ax[0].imshow(H,cmap='gray')
ax[0].set_title('Hue Channel')
ax[0].axis('off')
ax[1].imshow(S, cmap='gray')
ax[1].set_title('Saturation Channel')
ax[1].axis('off')
ax[2].imshow(V, cmap='gray')
ax[2].set_title('Value Channel')
ax[2].axis('off')
```

Out[270…
```
(np.float64(-0.5), np.float64(779.5), np.float64(437.5), np.float64(-0.5))
```



In [271…
```
sigma = 70
a = 0.8
```

```
x = np.arange(256)
t = x + a * 128 * np.exp(-(x - 128)**2 / (2 * sigma**2))
t = np.minimum(t, 255).astype(np.uint8)
S_transformed = t[S]
hsv_transformed = cv.merge([H, S_transformed, V])
vibrance_enhanced = cv.cvtColor(hsv_transformed, cv.COLOR_HSV2RGB)
```

In [272…

```
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
ax[0].set_title('Original Image')
ax[0].axis('off')

ax[1].imshow(vibrance_enhanced)
ax[1].set_title(f'Vibrance-Enhanced Image (a = {a})')
ax[1].axis('off')

ax[2].plot(x, t, color='purple', linewidth=2, label='Transformation')
ax[2].plot(x, x, '--', color='gray', label='Identity')
ax[2].set_title('Saturation Intensity Transformation')
ax[2].set_xlabel('Input Saturation')
ax[2].set_ylabel('Output Saturation')
ax[2].set_xlim(0, 255)
ax[2].set_ylim(0, 255)
ax[2].grid(alpha=0.3)
ax[2].legend()
fig.tight_layout()
plt.show()
```



The saturation plane contains high intensity values in colourful regions, such as the red areas of Spider-Man's suit. In contrast, low saturation values occur in nearly colourless regions, such as the buildings and sky. By increasing the saturation mainly in already colourful areas while preserving low-saturation regions, the image's vibrance is enhanced without making neutral areas look unnatural.

Question 5

In [273…

```
def histogram_equalization(f):

    hist = np.zeros(256, dtype=np.int32)
    for pixel in f.ravel():
```

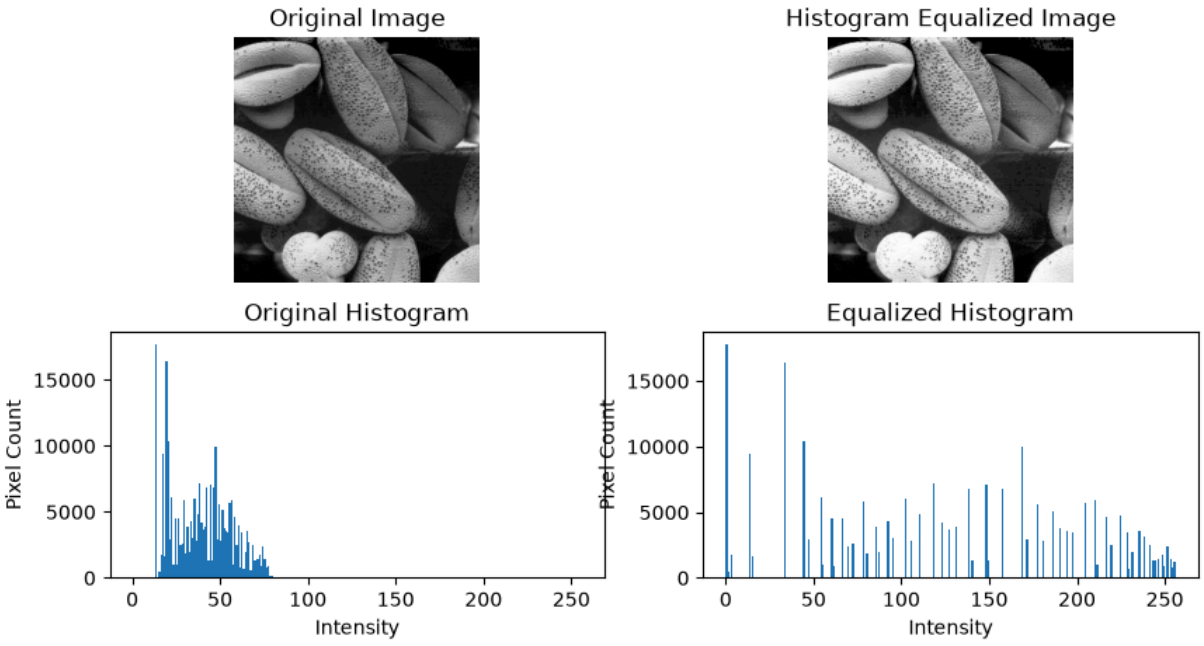
```
        hist[pixel] += 1
    cdf = np.cumsum(hist)
    cdf_min = cdf[cdf > 0][0]
    t = np.round(
        (cdf - cdf_min) /
        (f.size - cdf_min) * 255
    ).clip(0, 255).astype(np.uint8)
    return t[f]
f = cv.imread('images/shells.tif', cv.IMREAD_GRAYSCALE)
g = histogram_equalization(f)
```

In [274…

```
fig, ax = plt.subplots(2, 2, figsize=(10, 5))
ax[0, 0].imshow(f, cmap='gray')
ax[0, 0].set_title('Original Image')
ax[0, 0].axis('off')
ax[0, 1].imshow(g, cmap='gray')
ax[0, 1].set_title('Histogram Equalized Image')
ax[0, 1].axis('off')
ax[1,0].hist(f.ravel(), bins=256, range=(0, 256))
ax[1,0].set_title('Original Histogram')
ax[1,0].set_xlabel('Intensity')
ax[1,0].set_ylabel('Pixel Count')
ax[1,1].hist(g.ravel(), bins=256, range=(0, 256))
ax[1,1].set_title('Equalized Histogram')
ax[1,1].set_xlabel('Intensity')
ax[1,1].set_ylabel('Pixel Count')
```

Out[274…

```
Text(0, 0.5, 'Pixel Count')
```



As seen in the equalized histogram, the intensity values have been spread more evenly across the available intensity range, improving the overall contrast. However, some fluctuations are introduced due to quantization of the intensity values. The low-intensity peaks have remained relatively unchanged after equalization.

Question 6

In [275…
```
f = cv.imread('images/jennifer.jpeg')
hsv = cv.cvtColor(f, cv.COLOR_BGR2HSV)
H, S, V = cv.split(hsv)
threshold = 30
mask = np.uint8(S > threshold) * 255
```

In [276…
```
fig, ax = plt.subplots(1, 4, figsize=(16, 4))
ax[0].imshow(H, cmap='gray')
ax[0].set_title('Hue')
ax[1].imshow(S, cmap='gray')
ax[1].set_title('Saturation')
ax[2].imshow(V, cmap='gray')
ax[2].set_title('Value')
ax[3].imshow(mask, cmap='gray')
ax[3].set_title('Foreground Mask')
for a in ax: a.axis('off')
plt.show()
```

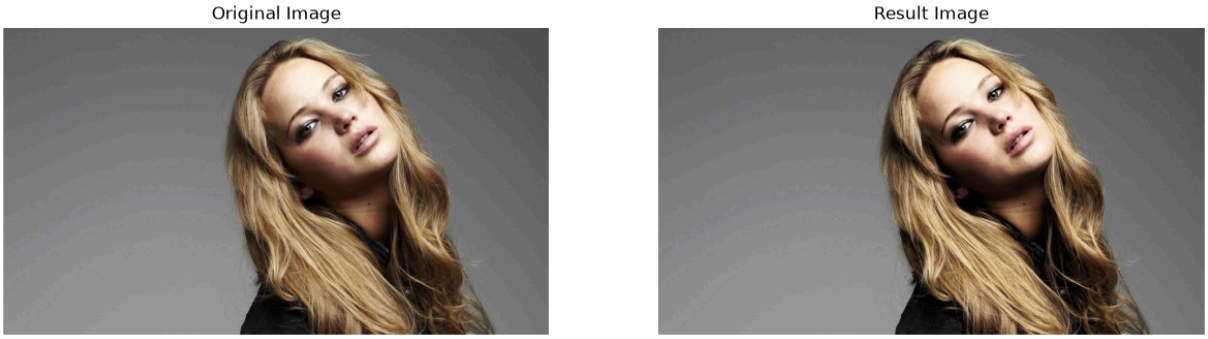


In [277…
```
foreground = cv.bitwise_and(f, f, mask=mask)
foreground_V = cv.bitwise_and(V, V, mask=mask)
hist = np.zeros(256, dtype=np.int32)
for pixel in foreground_V[mask > 0]:
    hist[pixel] += 1
cdf = np.cumsum(hist)
```

In [278…
```
cdf_min = cdf[cdf > 0][0]
t = np.round((cdf - cdf_min) / (cdf[-1] - cdf_min) * 255).astype(np.uint8)
equalized_V = t[V]
equalized_foreground_V = np.where(mask > 0,equalized_V,V).astype(np.uint8)
background_mask = cv.bitwise_not(mask)
background_V = cv.bitwise_and(V, V, mask=background_mask)
result_V = equalized_foreground_V.copy()
result_hsv = cv.merge([H, S, result_V])
result = cv.cvtColor( result_hsv, cv.COLOR_HSV2BGR)
```

In [279…
```
fig,ax= plt.subplots(1, 2, figsize=(15, 5))
ax[0].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
ax[0].set_title('Original Image')
ax[0].axis('off')
ax[1].imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB))
ax[1].set_title('Result Image')
ax[1].axis('off')
```

Out[279…
```
(np.float64(-0.5), np.float64(1599.5), np.float64(899.5), np.float64(-0.5))
```



The foreground was extracted using a threshold on the saturation plane. Histogram equalization was applied only to the value channel of the foreground, increasing the contrast of the subject while preserving the original background. The result demonstrates that histogram equalization can be selectively applied to a region of interest rather than the entire image.

Question 7

In [280…

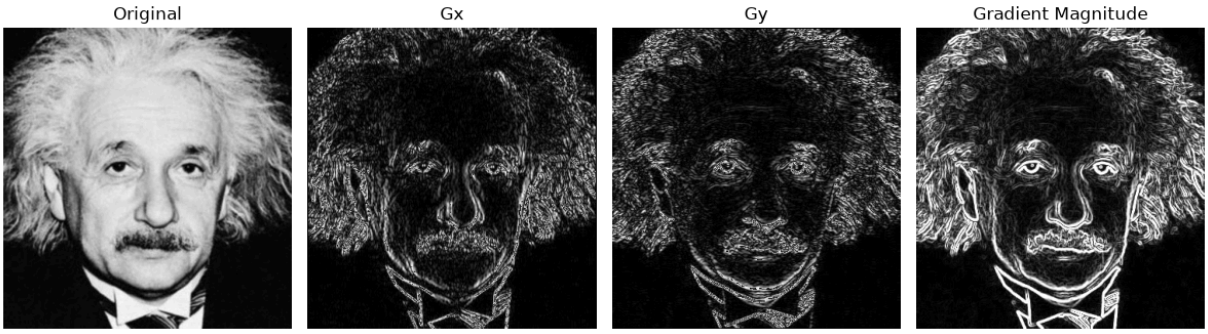
```
f = cv.imread('images/albert.jpeg', cv.IMREAD_GRAYSCALE)
Kx = np.array([
    [1, 0, -1],
    [2, 0, -2],
    [1, 0, -1]
])

Ky = np.array([
    [1, 2, 1],
    [0, 0, 0],
    [-1, -2, -1]
])
Gx = cv.filter2D(f, cv.CV_64F, Kx)
Gy = cv.filter2D(f, cv.CV_64F, Ky)
G = np.sqrt(Gx**2 + Gy**2)
Gx_display = np.uint8(np.abs(Gx))
Gy_display = np.uint8(np.abs(Gy))
G_display = np.uint8(np.clip(G, 0, 255))
```

In [281…

```
plt.figure(figsize=(12, 4))
plt.subplot(1, 4, 1)
plt.imshow(f, cmap='gray')
plt.title('Original')
plt.axis('off')
plt.subplot(1, 4, 2)
plt.imshow(Gx_display, cmap='gray')
plt.title('Gx')
plt.axis('off')
plt.subplot(1, 4, 3)
plt.imshow(Gy_display, cmap='gray')
plt.title('Gy')
plt.axis('off')
plt.subplot(1, 4, 4)
```

```
plt.imshow(G_display, cmap='gray')
plt.title('Gradient Magnitude')
plt.axis('off')
plt.tight_layout()
plt.show()
```



In [282…

```
def my_filter(img, kernel):
    h, w = img.shape
    output = np.zeros((h, w), dtype=np.float64)
    for i in range(1, h - 1):
        for j in range(1, w - 1):
            region = img[i-1:i+2, j-1:j+2]
            output[i, j] = np.sum(region * kernel)
    return output
Gx = my_filter(f, Kx)
Gy = my_filter(f, Ky)
G = np.sqrt(Gx**2 + Gy**2)
Gx_display = np.uint8(np.abs(Gx))
Gy_display = np.uint8(np.abs(Gy))
G_display = np.uint8(np.clip(G, 0, 255))
```

In [283…

```
plt.figure(figsize=(12, 4))

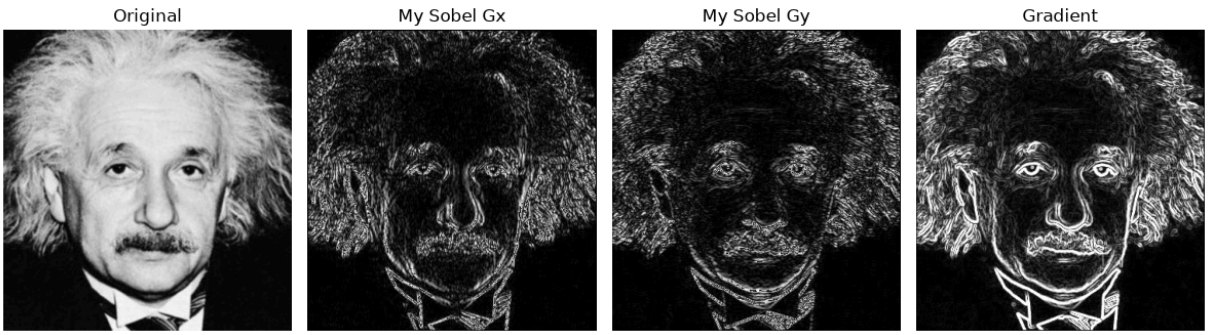
plt.subplot(1, 4, 1)
plt.imshow(f, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(Gx_display, cmap='gray')
plt.title('My Sobel Gx')
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(Gy_display, cmap='gray')
plt.title('My Sobel Gy')
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(G_display, cmap='gray')
plt.title('Gradient')
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```



In [284…

```
v_x = np.array([
    [1],
    [2],
    [1]
], dtype=np.float64)

h_x = np.array([
    [1, 0, -1]
], dtype=np.float64)

temp_x = cv.filter2D(f, cv.CV_64F, v_x)
Gx_c = cv.filter2D(temp_x, cv.CV_64F, h_x)
v_y = np.array([
    [1],
    [0],
    [-1]
], dtype=np.float64)

h_y = np.array([
    [1, 2, 1]
], dtype=np.float64)

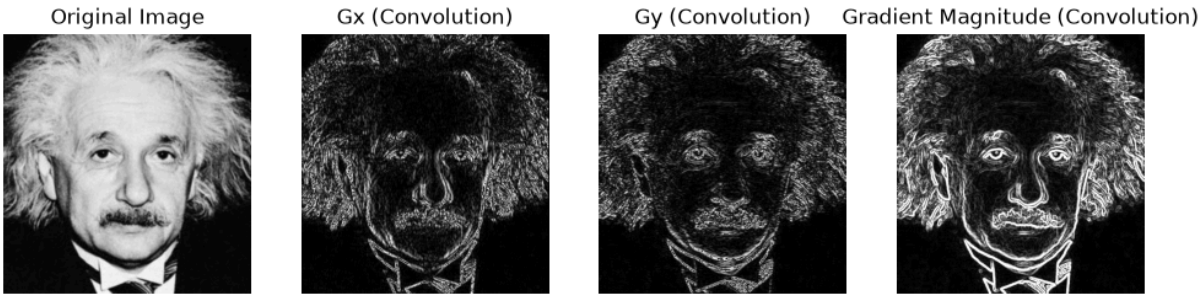
temp_y = cv.filter2D(f, cv.CV_64F, v_y)
Gy_c = cv.filter2D(temp_y, cv.CV_64F, h_y)

G_c = np.sqrt(Gx_c**2 + Gy_c**2)
```

In [285…

```
fig, ax = plt.subplots(1, 4, figsize=(12, 4))
ax[0].imshow(f, cmap='gray')
ax[0].set_title('Original Image')
ax[0].axis('off')
ax[1].imshow(np.uint8(np.abs(Gx_c)), cmap='gray')
ax[1].set_title('Gx (Convolution)')
ax[1].axis('off')
ax[2].imshow(np.uint8(np.abs(Gy_c)), cmap='gray')
ax[2].set_title('Gy (Convolution)')
ax[2].axis('off')
ax[3].imshow(np.uint8(np.clip(G_c, 0, 255)), cmap='gray')
ax[3].set_title('Gradient Magnitude (Convolution)')
ax[3].axis('off')
```

Out[285… (np.float64(-0.5), np.float64(363.5), np.float64(379.5), np.float64(-0.5))



The Sobel operator detects intensity changes in the horizontal and vertical directions. G_x emphasizes vertical edges while G_y emphasizes horizontal edges. The gradient magnitude combines both responses to represent the overall edge strength. The separable implementation produces the same Sobel operation using two 1-D filters.

Question 8

In [286…

```
def zoom_nearest_neighbor(img, scale):
    h, w = img.shape
    new_h, new_w = int(h * scale), int(w * scale)
    zoomed_img = np.zeros((new_h, new_w, ), dtype=img.dtype)
    for i in range(new_h):
        for j in range(new_w):
            orig_i = int(i / scale)
            orig_j = int(j / scale)
            zoomed_img[i, j] = img[orig_i, orig_j]
    return zoomed_img
def zoom_bilinear(img, scale):
    h, w= img.shape
    new_h, new_w = int(h * scale), int(w * scale)
    zoomed_img = np.zeros((new_h, new_w), dtype=img.dtype)
    for i in range(new_h):
        for j in range(new_w):
            orig_i = i / scale
            orig_j = j / scale
            i0, j0 = int(orig_i), int(orig_j)
            i1, j1 = min(i0 + 1, h - 1), min(j0 + 1, w - 1)
            di, dj = orig_i - i0, orig_j - j0
            zoomed_img[i, j] = (1 - di) * (1 - dj) * img[i0, j0] + \
                               (1 - di) * dj * img[i0, j1] + \
                               di * (1 - dj) * img[i1, j0] + \
                               di * dj * img[i1, j1]
    return zoomed_img
def zoom(img, scale, method='nearest'):
    if scale <= 0 or scale > 10:
        raise ValueError("Scale must be a positive number.")
    if method == 'nearest':
        B, G, R = cv.split(img)
        B_zoomed = zoom_nearest_neighbor(B, scale)
        G_zoomed = zoom_nearest_neighbor(G, scale)
        R_zoomed = zoom_nearest_neighbor(R, scale)
```

```
        return cv.merge([B_zoomed, G_zoomed, R_zoomed])
    elif method == 'bilinear':
        B, G, R = cv.split(img)
        B_zoomed = zoom_bilinear(B, scale)
        G_zoomed = zoom_bilinear(G, scale)
        R_zoomed = zoom_bilinear(R, scale)
        return cv.merge([B_zoomed, G_zoomed, R_zoomed])
    else:
        raise ValueError("Method must be 'nearest' or 'bilinear'.")
```

In [287…

```
f = cv.imread('images/im01small.png')
img1_nearest=zoom(f, 4, method='nearest')
img1_bilinear=zoom(f, 4, method='bilinear')

f= cv.imread('images/im02small.png')
img2_nearest = zoom(f, 4, method='nearest')
img2_bilinear = zoom(f, 4, method='bilinear')
```

In [288…

```
fig, ax = plt.subplots(2, 2, figsize=(10, 10))

ax[0, 0].imshow(cv.cvtColor(img1_nearest, cv.COLOR_BGR2RGB))
ax[0, 0].set_title('Image 1 - Nearest Neighbor')
ax[0, 0].axis('off')

ax[0, 1].imshow(cv.cvtColor(img1_bilinear, cv.COLOR_BGR2RGB))
ax[0, 1].set_title('Image 1 - Bilinear Interpolation')
ax[0, 1].axis('off')

ax[1, 0].imshow(cv.cvtColor(img2_nearest, cv.COLOR_BGR2RGB))
ax[1, 0].set_title('Image 2 - Nearest Neighbor')
ax[1, 0].axis('off')

ax[1, 1].imshow(cv.cvtColor(img2_bilinear, cv.COLOR_BGR2RGB))
ax[1, 1].set_title('Image 2 - Bilinear Interpolation')
ax[1, 1].axis('off')

plt.tight_layout()
plt.show()
```

Image 1 - Nearest Neighbor



Image 1 - Bilinear Interpolation



Image 2 - Nearest Neighbor



Image 2 - Bilinear Interpolation



In [289…

```
def normalized_ssd(img, ori_img):
    img = img.astype(np.float64)
    ori_img = ori_img.astype(np.float64)
    ssd = np.sum((img - ori_img) ** 2)
    normalized_ssd = ssd / np.sum(ori_img ** 2)
    return normalized_ssd

print("Normalized SSD for Image 1 - Nearest Neighbor:", normalized_ssd(img1_nearest
print("Normalized SSD for Image 1 - Bilinear Interpolation:", normalized_ssd(img1_b
print("Normalized SSD for Image 2 - Nearest Neighbor:", normalized_ssd(img2_nearest
print("Normalized SSD for Image 2 - Bilinear Interpolation:", normalized_ssd(img2_b
```

```
Normalized SSD for Image 1 - Nearest Neighbor: 0.012057947290248129
Normalized SSD for Image 1 - Bilinear Interpolation: 0.017718956455122437
Normalized SSD for Image 2 - Nearest Neighbor: 0.0041900419178234226
Normalized SSD for Image 2 - Bilinear Interpolation: 0.007757186831744027
```

For these two test images, nearest-neighbor interpolation produced a lower normalized SSD than bilinear interpolation. This indicates that the zoomed images and their corresponding originals were not necessarily generated in a way that makes bilinear interpolation reproduce the original more closely.

Question 9

In [290…

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

flower = cv.imread('images/flower.jpeg')
mask = np.zeros(flower.shape[:2], np.uint8)
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)
rect = (50, 100, 550, 490)
cv.grabCut(flower,mask,rect,bgdModel,fgdModel,5,cv.GC_INIT_WITH_RECT)
mask2 = np.where((mask == 2) | (mask == 0),0,1).astype('uint8')
foreground = flower * mask2[:, :, np.newaxis]
background = cv.subtract(flower, foreground)
```

In [291…

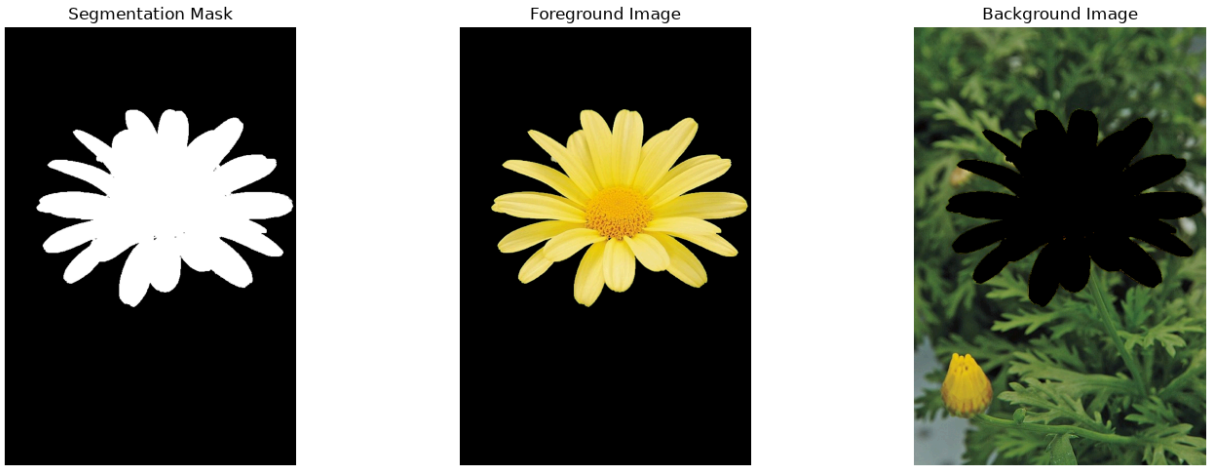
```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(mask2, cmap='gray')
plt.title('Segmentation Mask')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(cv.cvtColor(foreground, cv.COLOR_BGR2RGB))
plt.title('Foreground Image')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(cv.cvtColor(background, cv.COLOR_BGR2RGB))
plt.title('Background Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```



In [292…

```
background_blurred = cv.GaussianBlur(background,(51, 51),0)

enhanced = cv.add(foreground,background_blurred)
```

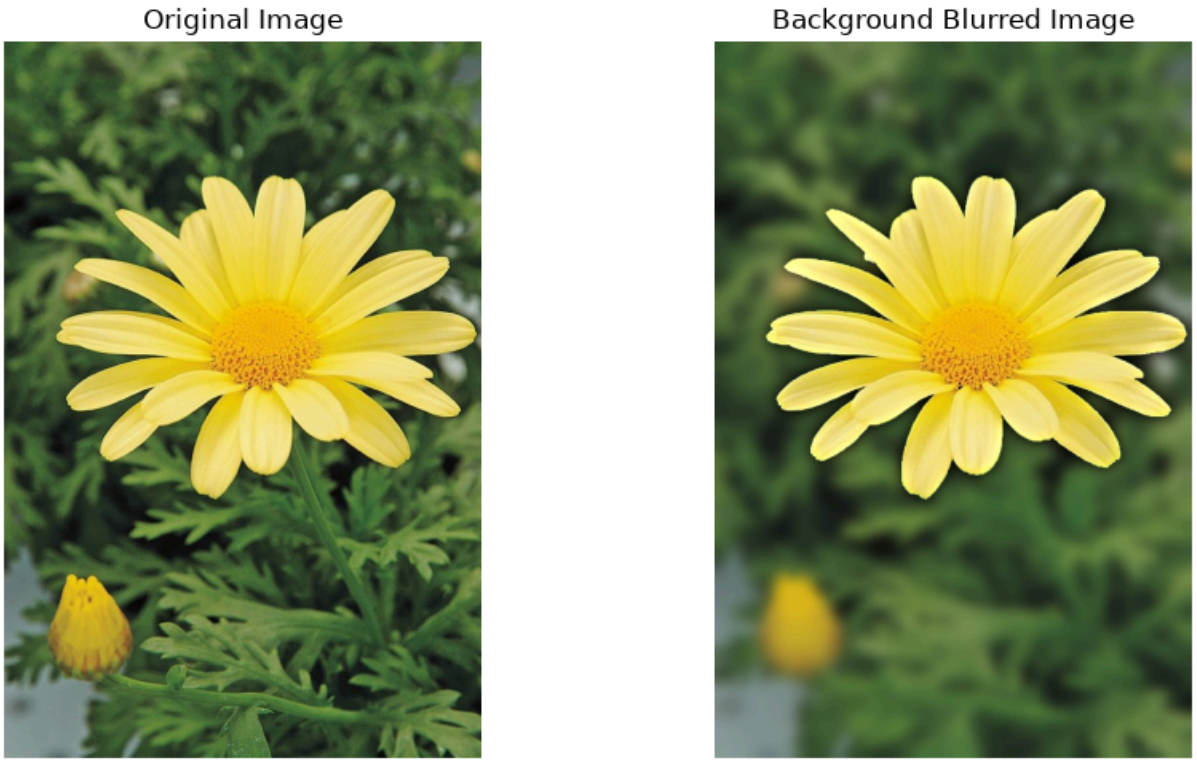
In [293…

```
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
plt.imshow(cv.cvtColor(flower, cv.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(cv.cvtColor(enhanced, cv.COLOR_BGR2RGB))
plt.title('Background Blurred Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```



Why is the background just beyond the flower dark?

The background image contains zero-valued pixels where the flower was removed. Gaussian filtering averages neighboring pixels, so near the flower boundary the background pixels are averaged with these zero-valued pixels. This makes the region immediately outside the flower darker.

Question 10

In [294…

```
f = cv.imread('images/lake.jpeg')
sigma_s = 3
sigma_r = 50
gaussian = cv.GaussianBlur(f,(9, 9),sigma_s)
bilateral = cv.bilateralFilter(f,9,sigma_r,sigma_s)
plt.figure(figsize=(15, 5))
```

```
plt.subplot(1, 3, 1)
plt.imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
plt.title('Original')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(cv.cvtColor(gaussian, cv.COLOR_BGR2RGB))
plt.title('Gaussian Blur')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(cv.cvtColor(bilateral, cv.COLOR_BGR2RGB))
plt.title('Bilateral Filter')
plt.axis('off')

plt.tight_layout()
plt.show()
```



In [295…

```
def my_bilateral_filter(img, d, sigma_color, sigma_space):

    img_float = img.astype(np.float64)

    h, w, c = img.shape

    filtered_img = np.zeros(
        (h, w, c),
        dtype=np.float64
    )

    half_d = d // 2

    for y in range(h):
        for x in range(w):

            wp_total = 0
            filtered_pixel = np.zeros(c)

            for ky in range(-half_d, half_d + 1):
                for kx in range(-half_d, half_d + 1):

                    ny = y + ky
                    nx = x + kx
```

```
                    if 0 <= ny < h and 0 <= nx < w:

                        # Spatial weight
                        spatial_weight = np.exp(
                            -(kx**2 + ky**2) /
                            (2 * sigma_space**2)
                        )

                        # Color difference
                        difference = (
                            img_float[y, x]
                            - img_float[ny, nx]
                        )

                        # Color/range weight
                        color_weight = np.exp(
                            -np.sum(difference**2) /
                            (2 * sigma_color**2)
                        )

                        # Total weight
                        weight = (
                            spatial_weight *
                            color_weight
                        )

                        filtered_pixel += (
                            weight *
                            img_float[ny, nx]
                        )

                        wp_total += weight

            filtered_img[y, x] = (
                filtered_pixel / wp_total
            )

    return np.clip(
        filtered_img,
        0,
        255
    ).astype(np.uint8)
```

In [296…

```
opencv_bilateral = cv.bilateralFilter(
    f,
    9,
    sigma_r,
    sigma_s
)

my_bilateral = my_bilateral_filter(
    f,
    9,
    sigma_r,
```

```
    sigma_s
)
```

In [297…

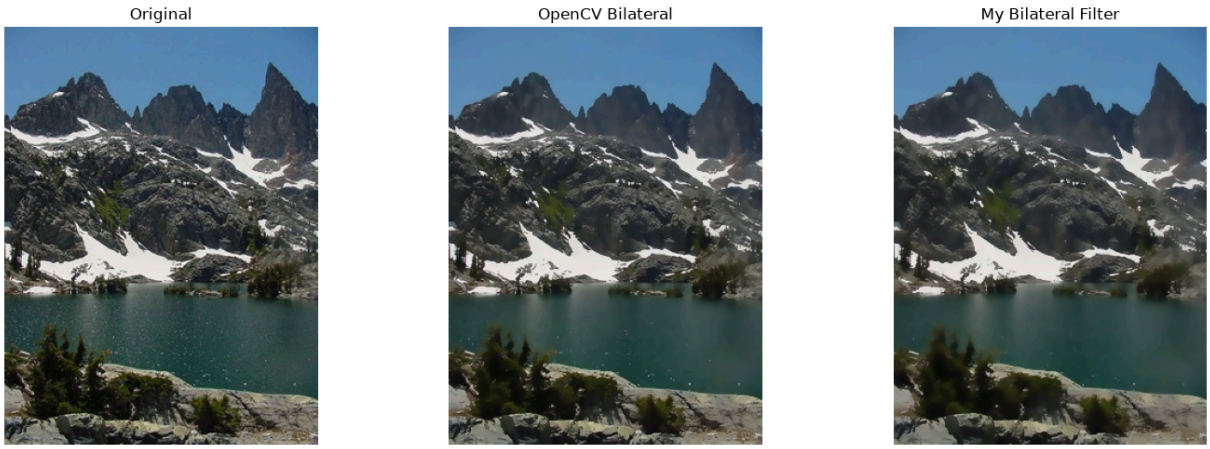
```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
plt.title('Original')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(cv.cvtColor(opencv_bilateral, cv.COLOR_BGR2RGB))
plt.title('OpenCV Bilateral')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(cv.cvtColor(my_bilateral, cv.COLOR_BGR2RGB))
plt.title('My Bilateral Filter')
plt.axis('off')

plt.tight_layout()
plt.show()
```



In [298…

```
def normalized_ssd(img1, img2):

    img1 = img1.astype(np.float64)
    img2 = img2.astype(np.float64)

    ssd = np.sum((img1 - img2)**2)

    return ssd / np.sum(img1**2)
```

In [299…

```
nssd = normalized_ssd(
    opencv_bilateral,
    my_bilateral
)

print("Normalized SSD:", nssd)
```

```
Normalized SSD: 0.001577938703972043
```

The Gaussian filter smooths the image but also blurs edges because pixels are averaged based mainly on spatial proximity. The bilateral filter produces smoothing while preserving edges by considering both spatial distance and color similarity. The own implementation produced a normalized SSD of approximately 0.00158 compared with OpenCV's bilateral filter, indicating that the two results are very similar.